`get_log_probs` — Step-by-Step Explanation

JR

February 28, 2026

Contents

1 Overview

`get_log_probs` is a utility function used during transformer training. Given the model's raw output (logits) and the actual token sequence, it returns the **log probability the model assigned to each correct next token**.

```python
def get_log_probs(
    logits: Float[Tensor, "batch posn d_vocab"],
    tokens: Int[Tensor, "batch posn"]
) -> Float[Tensor, "batch posn-1"]:
    log_probs = logits.log_softmax(dim=-1)
    log_probs_for_tokens = (
        log_probs[:, :-1]
            .gather(dim=-1, index=tokens[:, 1:].unsqueeze(-1))
            .squeeze(-1)
    )
    return log_probs_for_tokens
```

2 Inputs and Output

Argument	Shape	Meaning
`logits`	[batch, posn, d_{vocab}]	Raw unnormalized scores for every vocab token at every position
`tokens`	[batch, posn]	Actual token IDs in the sequence
return	[batch, posn − 1]	Log prob of the correct next token at each position

3 The Core Alignment Idea

A transformer at position i predicts the token at position $i + 1$. This means the model's output at position i must be evaluated against the *next* token in the sequence, not the current one.

Consider the sequence `["The", "cat", "sat", "."]` with token IDs $[1, 2, 3, 4]$:

Position	0	1	2	3
Token	`"The"`	`"cat"`	`"sat"`	`"."`
Model predicts →	`"cat"`	`"sat"`	`"."`	*(no target)*

The last position has no ground-truth next token to score against, so the output has shape posn − 1 rather than posn.

4 How the Two Slices Work

4.1 `log_probs[:, :-1]` — Slicing a Three-Dimensional Tensor

`log_probs` has shape [batch, posn, d_{vocab}] — three axes. The notation `[:, :-1]` supplies only **two** explicit index expressions. Python and PyTorch apply a simple rule:

> *Any trailing dimension not mentioned in the index expression receives an implicit `:` — meaning "keep every element along that axis".*

Therefore `log_probs[:, :-1]` is *exactly* equivalent to `log_probs[:, :-1, :]`. The three index slots map to axes as follows:

Slot	Value	Axis	Effect
1st	`:`	dim 0 — batch	Keep every batch item (unchanged)
2nd	`:-1`	dim 1 — posn	Keep indices $0, 1, \ldots,$ posn $- 2$; *drop* the last position
3rd	: (implicit)	dim 2 — d_{vocab}	Keep *every* vocabulary entry (unchanged)

Shape: [**batch**, **posn**, $d_{\textbf{vocab}}$] $\longrightarrow$ [**batch**, **posn** -1, $d_{\textbf{vocab}}$]

The critical insight is that the third dimension is **entirely untouched**. The slice only removes one "slice of pages" along the position axis. For every surviving (batch, posn) pair the full d_{vocab}-wide row of log probabilities is preserved intact. Visualise `log_probs` as a stack of batch matrices, each of size posn $\times\, d_{\text{vocab}}$. The operation removes the *last row* from every matrix while leaving each row's full width untouched:

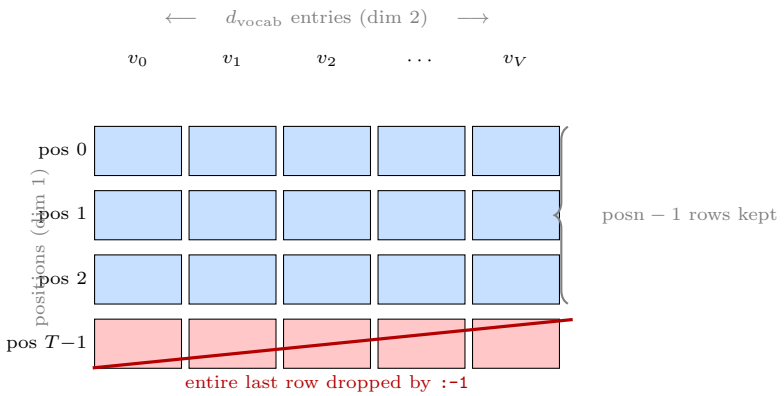

Figure 1: Effect of `log_probs[:, :-1]` on one batch item. The slice operates *only* on dim 1 (position axis): it discards the last row (pos $T-1$, red). The entire width of dim 2 — all d_{vocab} log-probability entries — is preserved for every kept row (blue). Dim 0 (batch) is also untouched; the same operation applies identically to every item in the batch.

4.2 `tokens[:, 1:]` — Slicing a Two-Dimensional Tensor

`tokens` has shape [batch, posn] — two axes. The slice `[:, 1:]` maps to:

Slot	Value	Axis	Effect
1st	`:`	dim 0 — batch	Keep every batch item
2nd	`1:`	dim 1 — posn	Keep indices $1, 2, \ldots,$ posn $- 1$; *drop* index 0 (the first token)

Shape: [**batch**, **posn**] $\longrightarrow$ [**batch**, **posn** -1]

The slice expression `1:` is standard Python *start-from-1-to-end* notation. In general `a:b` selects elements whose indices satisfy $a \le i < b$; omitting the upper bound means "go all the way to the end". So `1:` retains every index ≥ 1, dropping only index 0.

The semantic effect is a **left-shift by one position**: element i in the result is the element that was at position $i + 1$ in the original tensor. Concretely, for the sequence [`"The"`, `"cat"`, `"sat"`, `"."`] with token IDs $[1, 2, 3, 4]$:

	Position in dim 1			
	0	1	2	3
Before `tokens`	`"The"` (1)	`"cat"` (2)	`"sat"` (3)	`"."` (4)
After `tokens[:, 1:]`	dropped	`"cat"` (2) →index 0	`"sat"` (3) →index 1	`"."` (4) →index 2

The first token `"The"` is dropped because it is the starting token — no earlier model output exists that should have predicted it as a *next* token. What remains is the list of **ground-truth targets**: exactly the token each position $0, 1, \ldots, \text{posn} - 2$ was supposed to predict.

Why the shapes now match. After both slices:

$$\underbrace{\texttt{log\_probs[:, :-1]}}_{\text{shape [batch, posn}-1, d_{\text{vocab}}]} \quad \text{and} \quad \underbrace{\texttt{tokens[:, 1:]}}_{\text{shape [batch, posn}-1]}$$

Both have the same [batch, posn $-$ 1] prefix. Position i in `log_probs[:, :-1]` holds the distribution the model produced at position i, and position i in `tokens[:, 1:]` holds the correct token the model should have predicted at that position — namely the token originally at position $i + 1$.

5 Diagram 1 — Sequence Alignment and Slicing

The two slicing operations (`log_probs[:, :-1]` and `tokens[:, 1:]`) create a matched pair of tensors: every prediction position lines up with its ground-truth target. Red cells are discarded; blue cells are prediction blocks; green cells are target tokens.

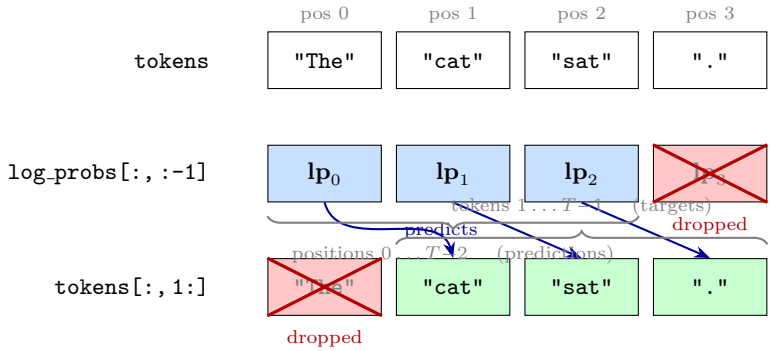

Figure 2: Slicing alignment. Each blue prediction block $\mathbf{lp}_i$ is evaluated against the green target token at position $i{+}1$. The red cells (last prediction, first token) are discarded because they have no counterpart.

6 Diagram 2 — The `gather` Operation

After slicing, `log_probs[:, :-1]` is a matrix of shape [posn$-$1, d_{vocab}]. `gather` selects **one cell per row** — the log probability of the actual next token — and assembles them into the output vector.

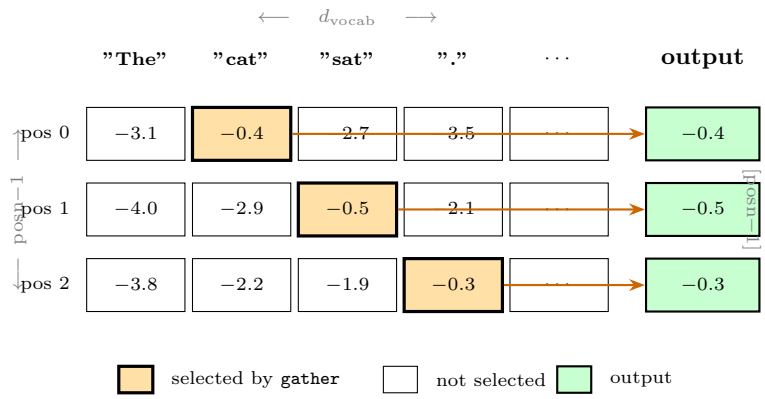

Figure 3: The `gather` operation. Each row of `log_probs[:, :-1]` holds log probabilities over all d_{vocab} tokens. `gather` picks the **highlighted** (orange) cell — the log prob of the true next token — and writes it to the output vector. The final `squeeze(-1)` collapses the trailing size-1 dimension, yielding shape [batch, posn−1].

7 Step-by-Step Walkthrough

7.1 Step 1 — Convert logits to log probabilities

```
log_probs = logits.log_softmax(dim=-1)
# shape: [batch, posn, d_vocab]  (unchanged)
```

`log_softmax` is applied along `dim=-1` (the vocab dimension). For each position p in batch item b, it computes:

$$\text{log\_probs}[b,p,v] = \log \frac{e^{\text{logits}[b,p,v]}}{\sum_{v'} e^{\text{logits}[b,p,v']}} = \log P(\text{token}_v \mid \text{context up to } p)$$

Using `log_softmax` instead of `softmax + log` is numerically stable because it avoids computing very small probabilities before taking the log.

7.2 Step 2 — Drop the last position

```
log_probs[:, :-1]
# shape: [batch, posn-1, d_vocab]
```

The model at the last position has no next token to be evaluated against, so we discard it. Only positions $0, 1, \ldots, \text{posn} - 2$ remain.

7.3 Step 3 — Get the actual next tokens

```
tokens[:, 1:]
# shape: [batch, posn-1]
```

We drop the first token and keep the rest. These are the ground-truth *targets*: the token at index 1 is what position 0 should have predicted, the token at index 2 is what position 1 should have predicted, and so on.

The alignment is now:

$$\underbrace{\text{log\_probs}[:, :-1]}_{\text{predictions from positions } 0\ldots\text{posn}-2} \longleftrightarrow \underbrace{\text{tokens}[:, 1:]}_{\text{targets at positions } 1\ldots\text{posn}-1}$$

7.4 Step 4 — Add a trailing dimension for `gather`

```
tokens[:, 1:].unsqueeze(-1)
# shape: [batch, posn-1, 1]
```

`torch.gather` requires the *index* tensor to have the same number of dimensions as the *source* tensor. The source `log_probs[:, :-1]` has 3 dimensions $[\text{batch}, \text{posn}-1, d_{\text{vocab}}]$, so we add a trailing dimension of size 1 to the index tensor.

7.5 Step 5 — Select the log prob for the correct token

```
log_probs[:, :-1].gather(dim=-1, index=tokens[:, 1:].unsqueeze(-1))
# shape: [batch, posn-1, 1]
```

`gather(dim=-1, index=idx)` picks, for each (b, p) slot, exactly one value along the vocab dimension:

$$\text{output}[b, p, 0] = \text{log\_probs}[b,\ p,\ \underbrace{\text{tokens}[b,\ p+1]}_{\text{true next token ID}}] = \log P(\text{true next token} \mid \text{context})$$

7.6 Step 6 — Remove the trailing size-1 dimension

```
.squeeze(-1)
# shape: [batch, posn-1]
```

`squeeze(-1)` removes the trailing dimension of size 1 introduced by `unsqueeze` and `gather`, giving the clean output shape.

8 Concrete Example

Let batch = 1, sequence = [`"The"`, `"cat"`, `"sat"`], token IDs = $[1, 2, 3]$.

Tensor	What it contains
`log_probs`	shape $[1, 3, d_{\text{vocab}}]$; log probs at all 3 positions
`log_probs[:, :-1]`	shape $[1, 2, d_{\text{vocab}}]$; positions 0 and 1 only
`tokens[:, 1:]`	$[[2, 3]]$; "cat" and "sat" are the targets
`gather` output	picks `log_probs[0,0,2]` and `log_probs[0,1,3]`
result	$\big[\log P(\texttt{"cat"} \mid \texttt{"The"}),\ \log P(\texttt{"sat"} \mid \texttt{"The cat"})\big]$

9 Why Log Probabilities?

- **Numerical stability.** Raw probabilities of long sequences become vanishingly small; working in log space avoids underflow.
- **Summability.** The log probability of an entire sequence factorises as a sum:

$$\log P(x_1, x_2, \ldots, x_T) = \sum_{t=1}^{T} \log P(x_t \mid x_{<t})$$

- **Direct connection to cross-entropy loss.** Averaging the negated log probs over the sequence gives the standard language-modelling cross-entropy loss:

$$\mathcal{L} = -\frac{1}{T-1} \sum_{t=0}^{T-2} \log P(x_{t+1} \mid x_{\leq t})$$